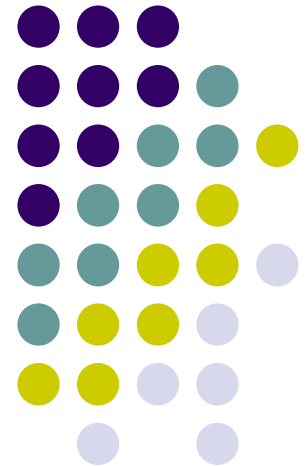
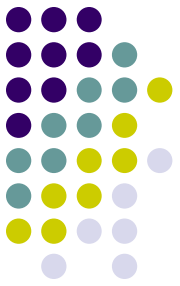


APPLICATION ARCHITECTURE

DoSE - CICT





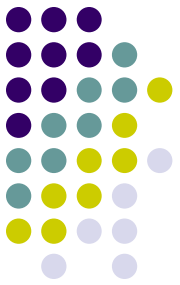
Application architecture

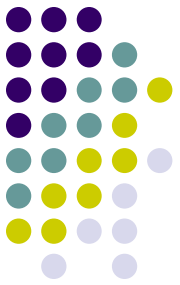
Application architecture

- is the underlying structure that developers work with to build an application.
- describes the tools and techniques that need to be used to ensure that the app is well designed and functioning at the highest level.

Application architecture

- Web Application Architecture
- Mobile Application Architecture

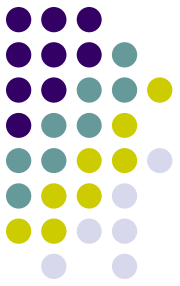




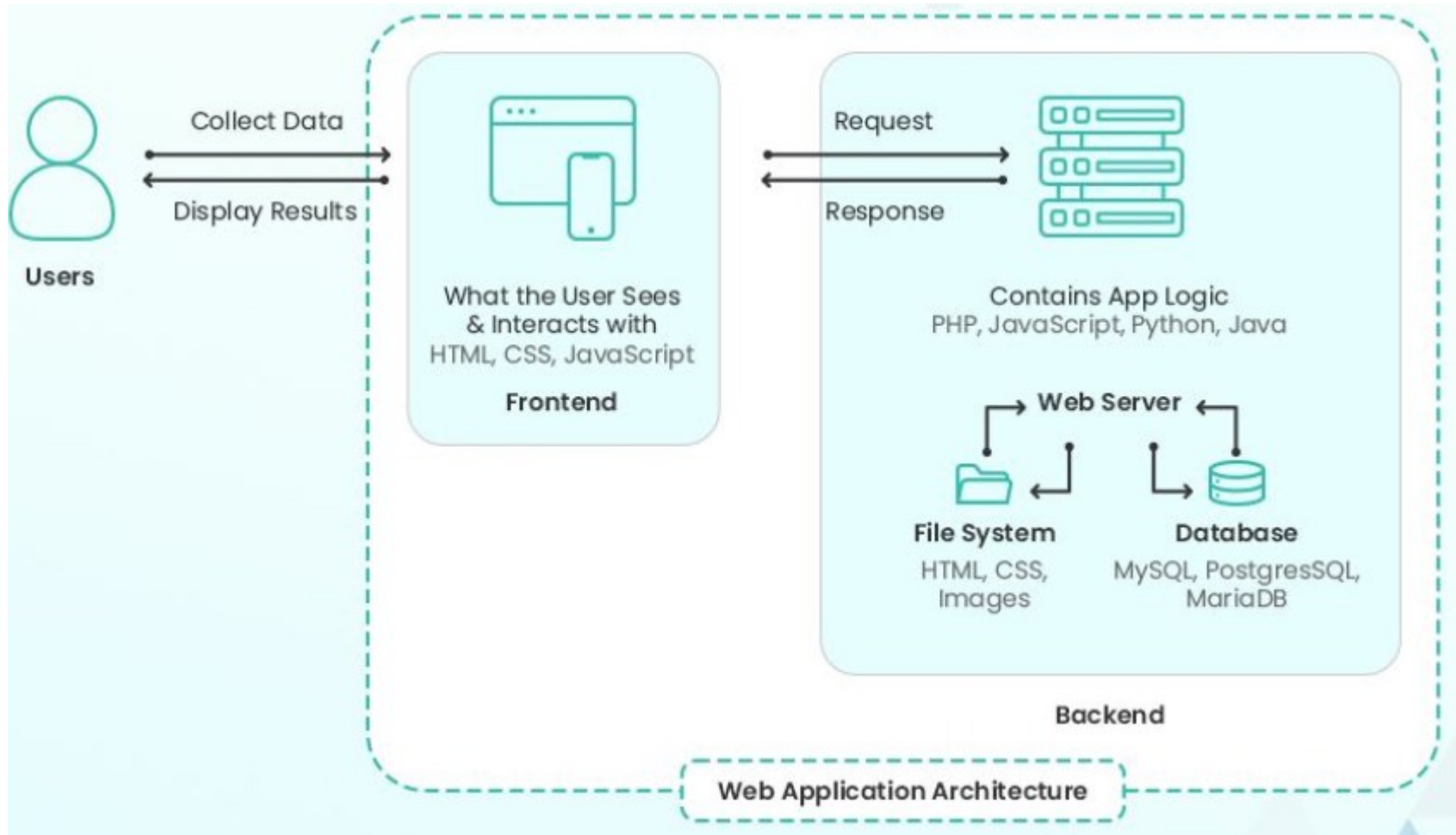
Web Application architecture

- **Web application architecture** is a “skeleton” or layout that displays the interactions between application components, middleware systems, user interfaces, and databases. This kind of interaction allows a number of applications to work together.

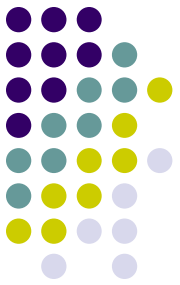
Web Application Architecture



- Web application architecture [diagram](https://radixweb.com/blog/web-application-architecture-guide)



Web Application Architecture

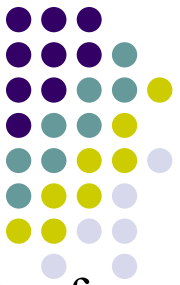


- **Fundamentals** of web application architecture
 - The client-side code is written in HTML, CSS, JavaScript and stored within the browser. **Client-side** (the frontend) is where user interaction takes place.
 - The server-side code is written in Java, PHP, Ruby, Python, etc. **Server-side** (the backend) controls the business logic and responds to HTTP requests.

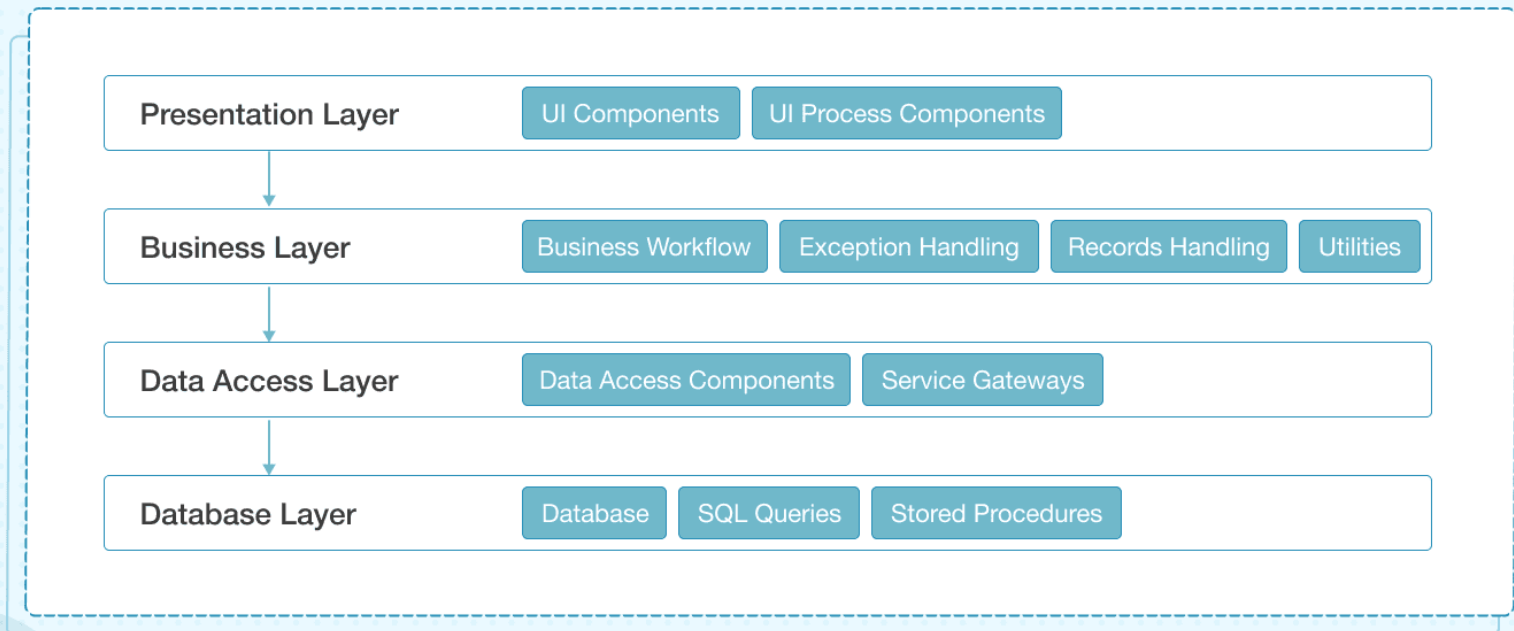


<https://www.intuz.com/guide-on-web-app-architecture>

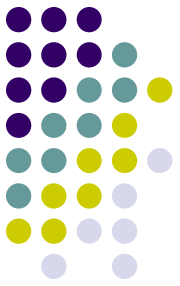
Web Application Architecture



- **Layers** of web application architecture
 - Modern web applications follow a layered architecture that consists of presentation, business, persistence/data access, and database layers.
 - Small applications have three layers, where, in some cases, the business and persistence layers act as one layer.
 - Complex applications have five or six layers.

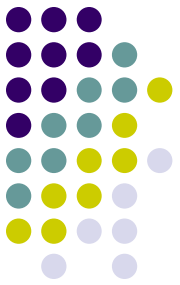


Web Application Architecture

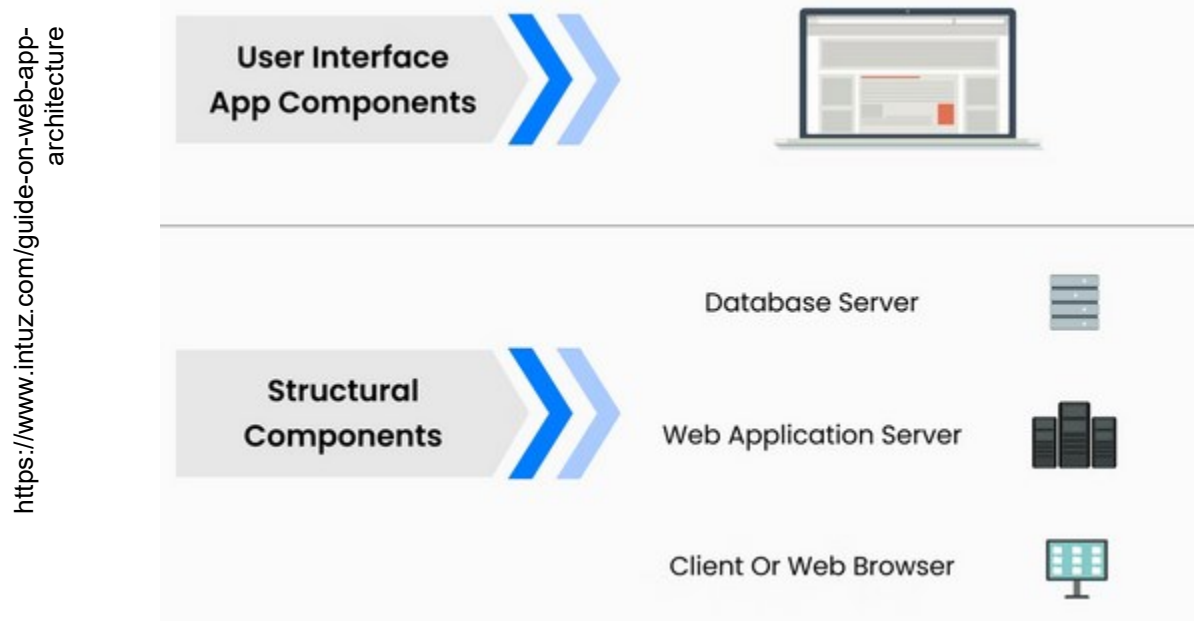


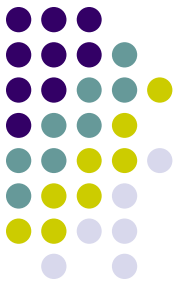
- **Layers** of web application architecture
 - **Presentation Layer** enables communication between the interface and browser to facilitate user interaction.
 - **Business Layer** defines the business logic and rules. It processes browser requests, executes the business logic associated with the requests, and then sends it to the presentation layer.
 - **Persistence Layer (Data Access Layer)** is responsible for data persistence. It's closely connected to the business layer and has a database server that retrieves data from corresponding servers. It comprises various third-party integrations like Payment gateways, social logins, etc.
 - **Database Layer (Data Service Layer)** holds all the data and ensures data security by separating the business logic from the client-side.

Web Application Architecture



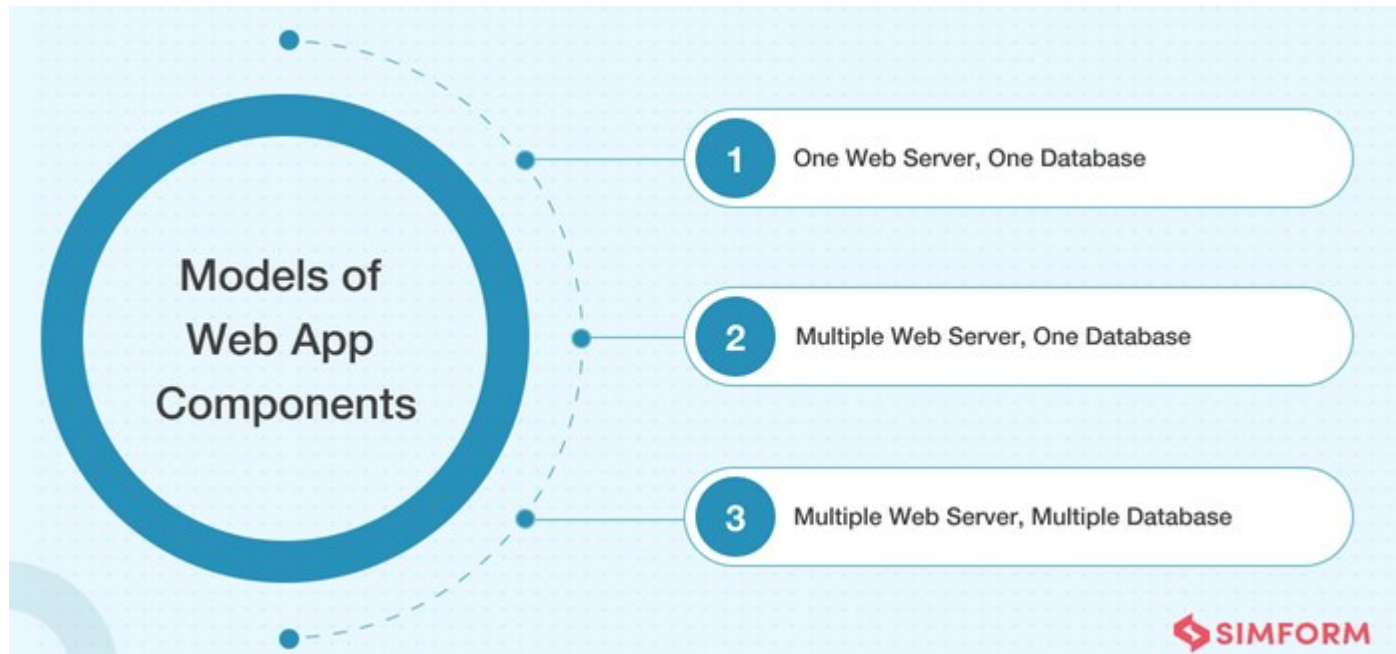
- **Components** of web application architecture
 - **User interface components** are a part of the visual interface of a web application and have no interaction with the architecture. They are responsible for creating the visuals of a web page and lay the foundation for user experience. They consist of activity logs, configuration settings, dashboards, statistics, widgets, notifications, etc.
 - **Structural components** consist of client and server components. Client components exist in the user's browser and interact with the functionality of web applications. Server components are bifurcated into a web app server that handles business logic and a database server that stores persistent data.



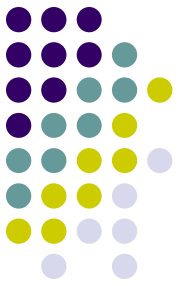


Web Application Architecture

- Models of web application architecture



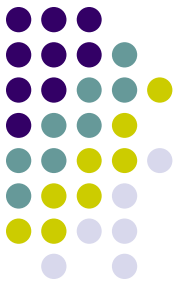
<https://www.simform.com/blog/web-application-architecture/>



Web Application Architecture

- Some **types** of web application architecture
 - Single Page Application Architecture
 - Microservice Architecture
 - Serverless Architecture
 - Progressive Web Applications

Web Application Architecture

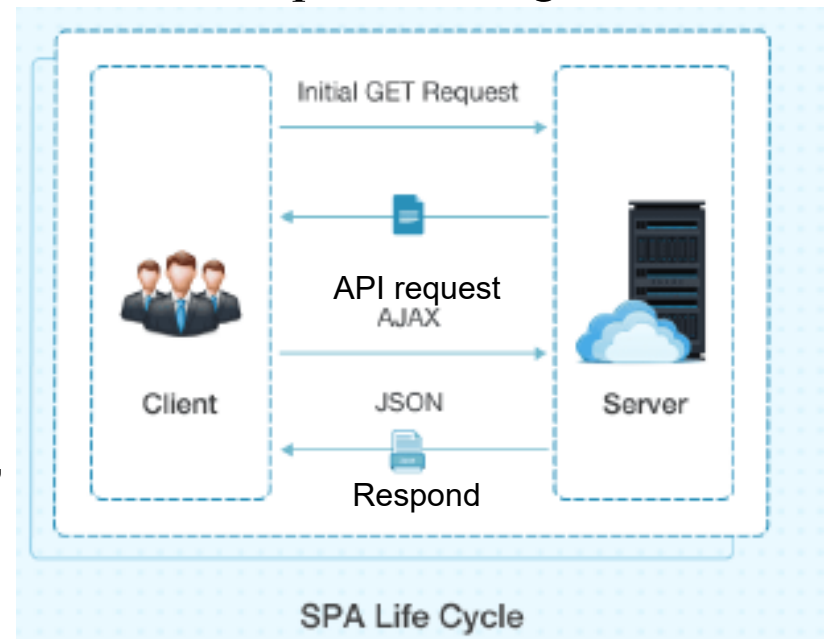


- **Single Page Application (SPA) Architecture**

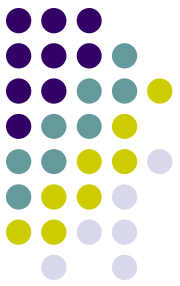
- Instead of loading a new page, SPAs load a single web page and reload the requested data on the same page with dynamically updated content. The rest of the web page remains untouched.
- They are developed on the client-side using JavaScript frameworks.
- Examples of a SPA: Facebook, Gmail, Google Drive, and Twitter.
- If a mobile app is needed, no additional efforts are required for API development – the mobile app could use the same API as Web.
- Drawbacks: the long time needed for the first load, poor routing and limited support of outdated browsers.

Client: Presentation layer

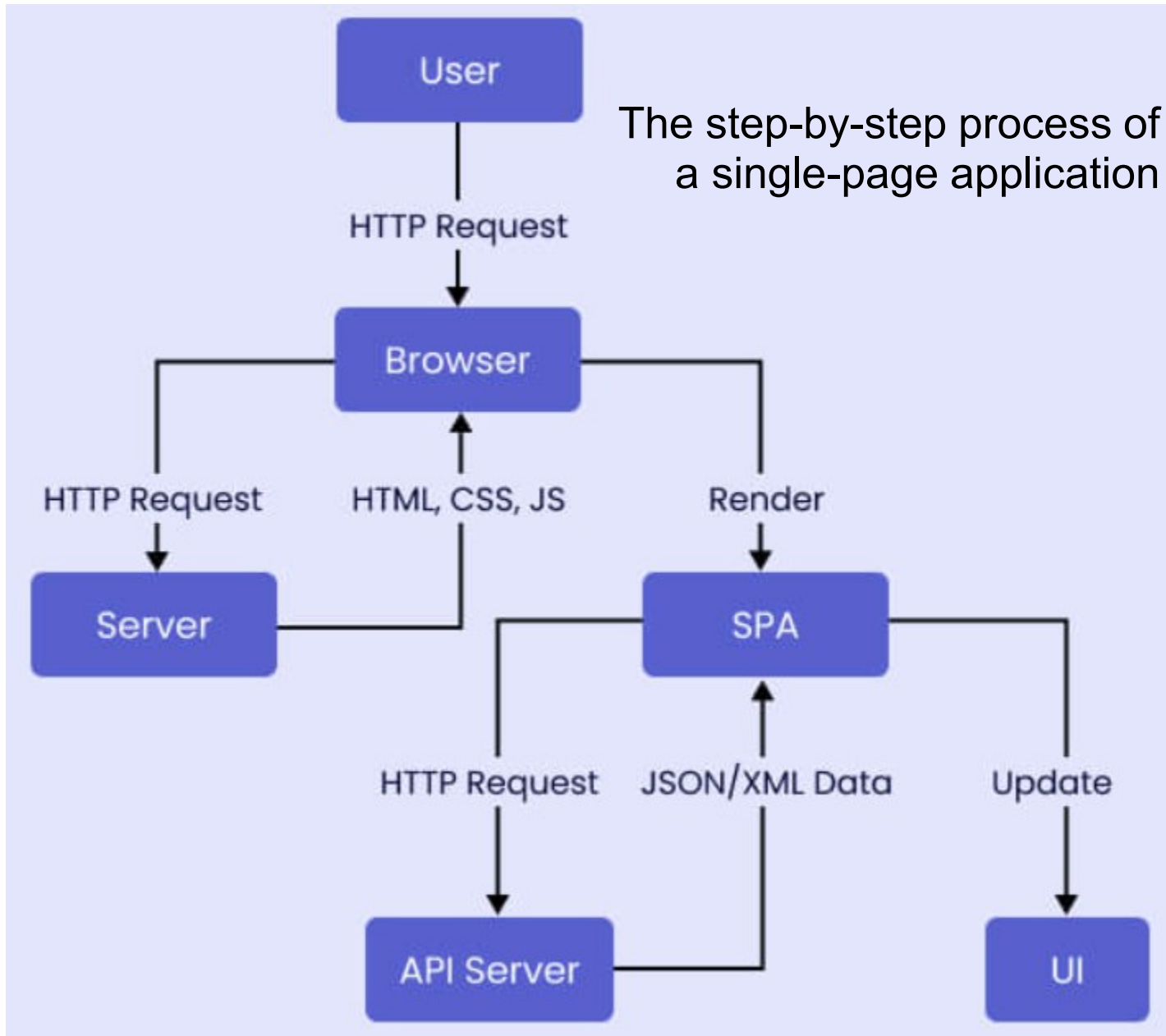
Server: Business, Data Access,
and Database layers



- Single Page Application (SPA) Architecture

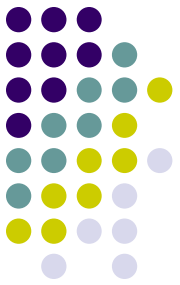


The step-by-step process of a single-page application



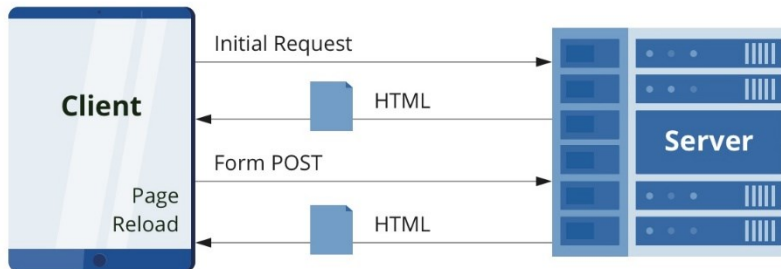
<https://www.spaceotechnologies.com/glossary/web-app/what-is-single-page-application/>

Web Application Architecture

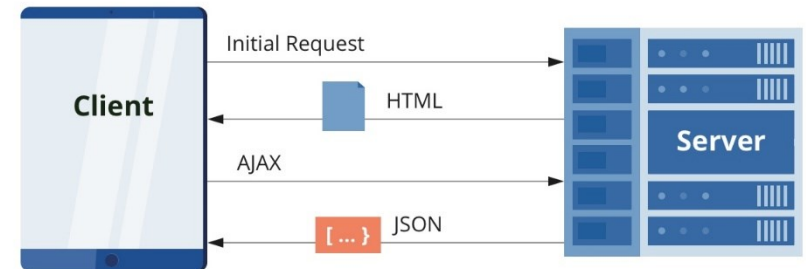


- Single Page Application (SPA) Architecture

Traditional Page Lifecycle



SPA Lifecycle



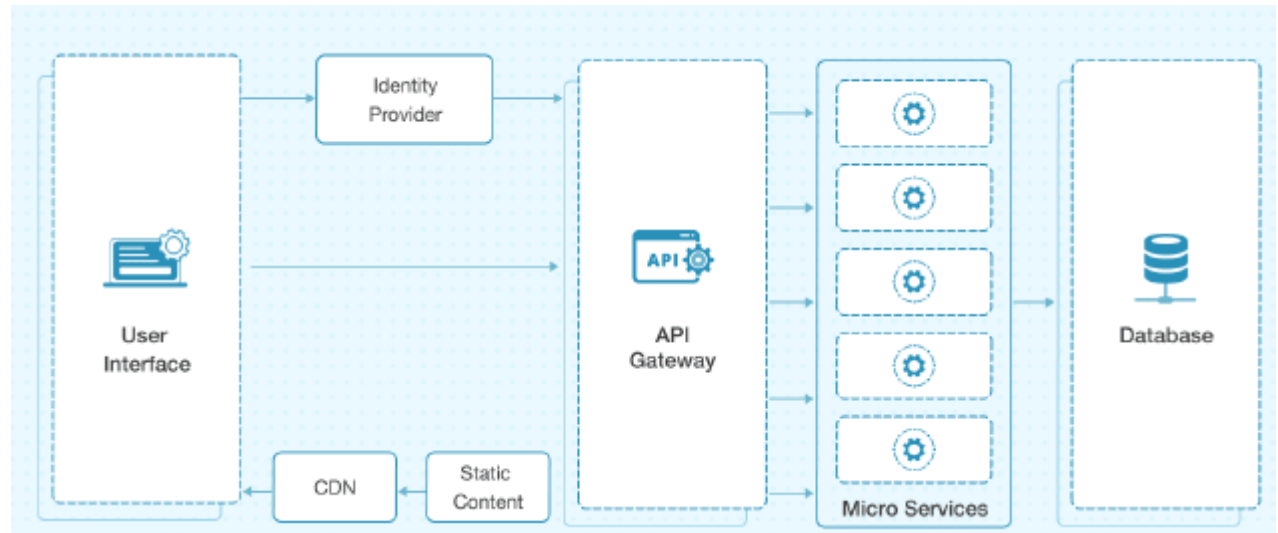
<https://dzone.com/articles/the-comparison-of-single-page-and-multi-page-appli>

Web Application Architecture



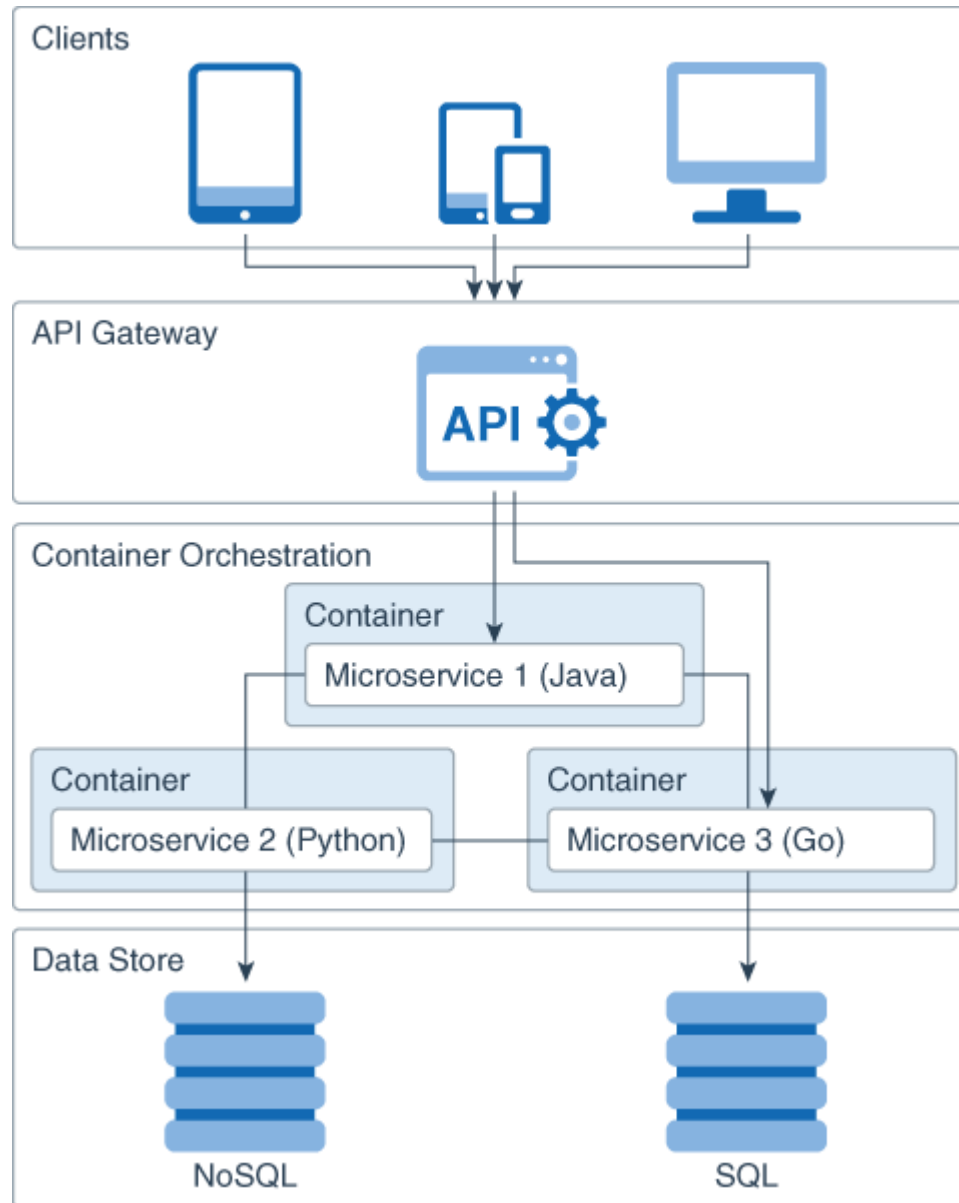
- **Microservice Architecture** (one of the types of SOA – service-oriented architecture)
 - Each microservice deals with small and lightweight services and execute a single functionality. It helps support multiple platforms and devices across the cloud, IoT, and mobile.
 - The services are loosely coupled to be developed sometimes in a different programming language, tested, maintained, and deployed independently. Such services can also communicate with other services through APIs to solve complex business problems.
 - Amazon, Netflix, SoundCloud, Comcast, and eBay use microservices architecture.

CDN or Content Delivery System deals with sending HTML files, CSS files, JavaScript files, and images. It delivers the content of the end server throughout the world, so people can load various sources.

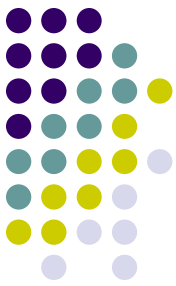


Web Application Architecture

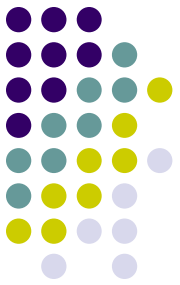
- Microservice Architecture



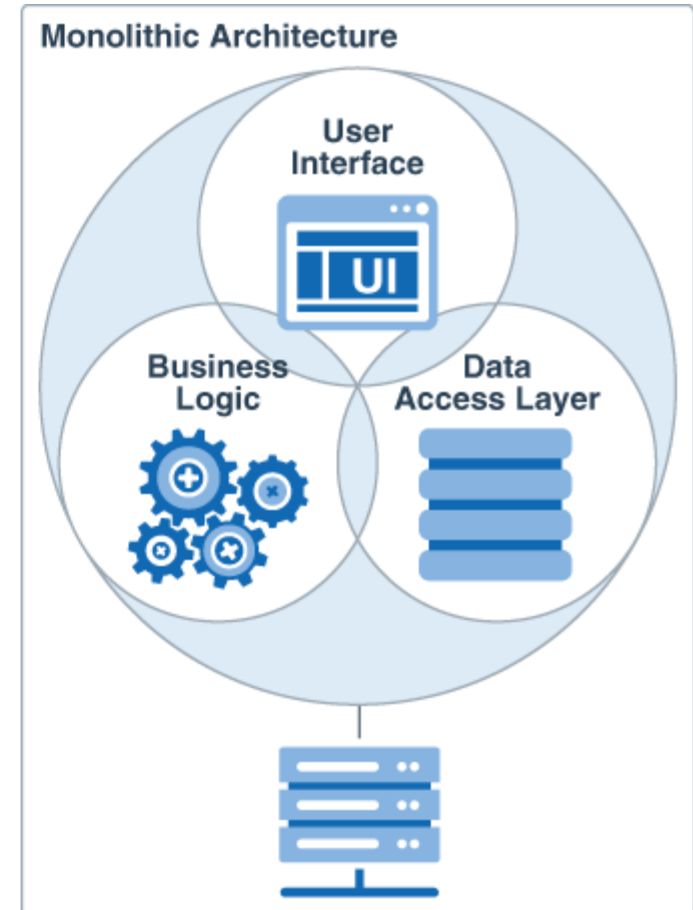
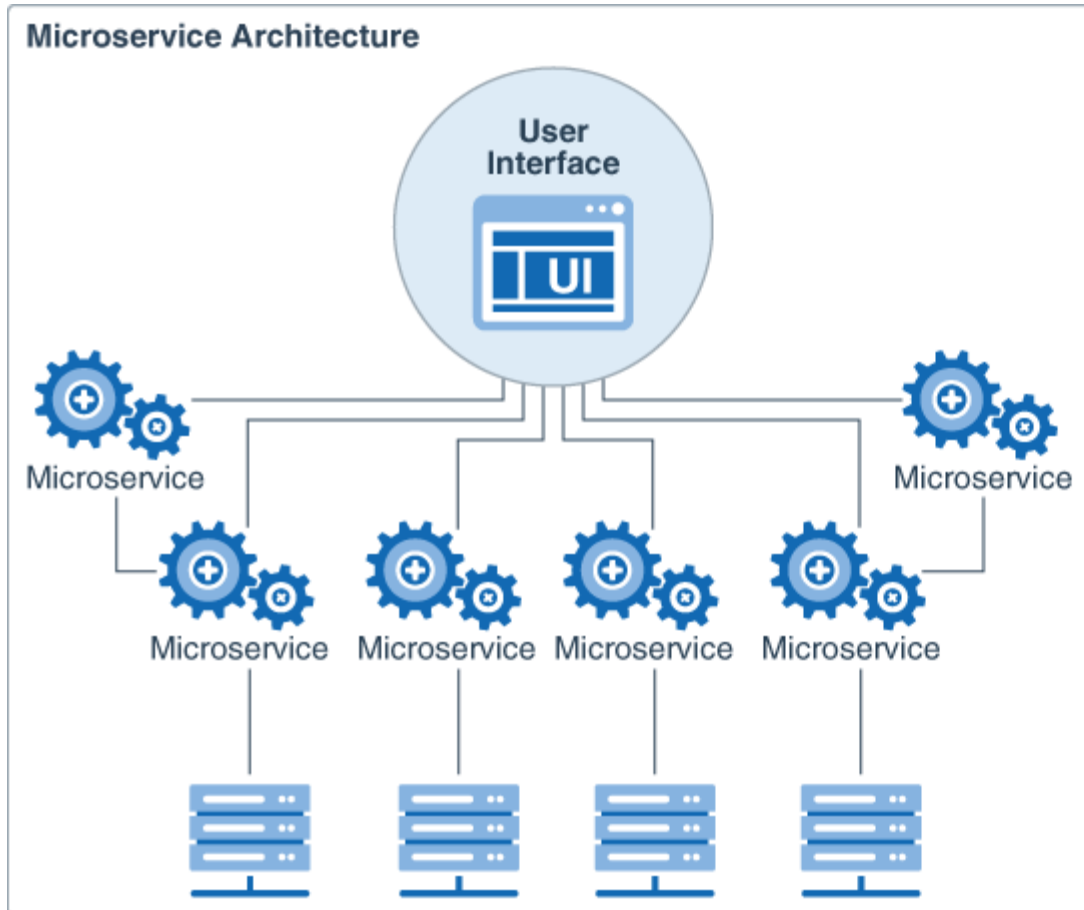
<https://docs.oracle.com/en/solutions/learn-architecture/microservice/index.html#GUID-BDCEFE30-C883-45D5-B2E6-325C241388A5>



Web Application Architecture



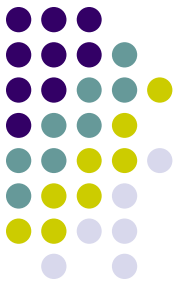
- **Microservice Architecture**



Characteristic	Microservices Architecture	Monolithic Architecture
Unit design	The application consists of loosely coupled services. Each service supports a single business task.	The entire application is designed, developed, and deployed as a single unit.
Functionality reuse	Microservices define APIs that expose their functionality to any client. The clients could even be other applications.	The opportunity for reusing functionality across applications is limited.
Communication within the application	To communicate with each other, the microservices of an application use the request-response communication model. The typical implementation uses REST API calls based on the HTTP protocol.	Internal procedures (function calls) facilitate communication between the components of the application. There is no need to limit the number of internal procedure calls.
Technological flexibility	Each microservice can be developed using a programming language and framework that best suits the problem that the microservice is designed to solve.	Usually, the entire application is written in a single programming language.
Data management	Decentralized: Each microservice may use its own database.	Centralized: The entire application uses one or more databases.

Characteristic	Microservices Architecture	Monolithic Architecture
Deployment	Each microservice is deployed independently, without affecting the other microservices in the application.	Any change, however small, requires redeploying and restarting the entire application.
Maintainability	Microservices are simple, focused, and independent. So the application is easier to maintain.	As the application scope increases, maintaining the code becomes more complex.
Resiliency	The application functionality is distributed across multiple services. If a microservice fails, the functionality offered by the other microservices continues to be available.	A failure in any component could affect the availability of the entire application.
Scalability	Each microservice can be scaled independently of the other services.	The entire application must be scaled, even when the business requirement is for scaling only certain parts of the application.

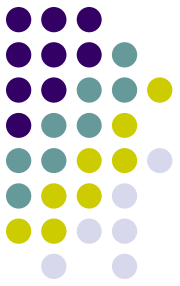
Web Application Architecture



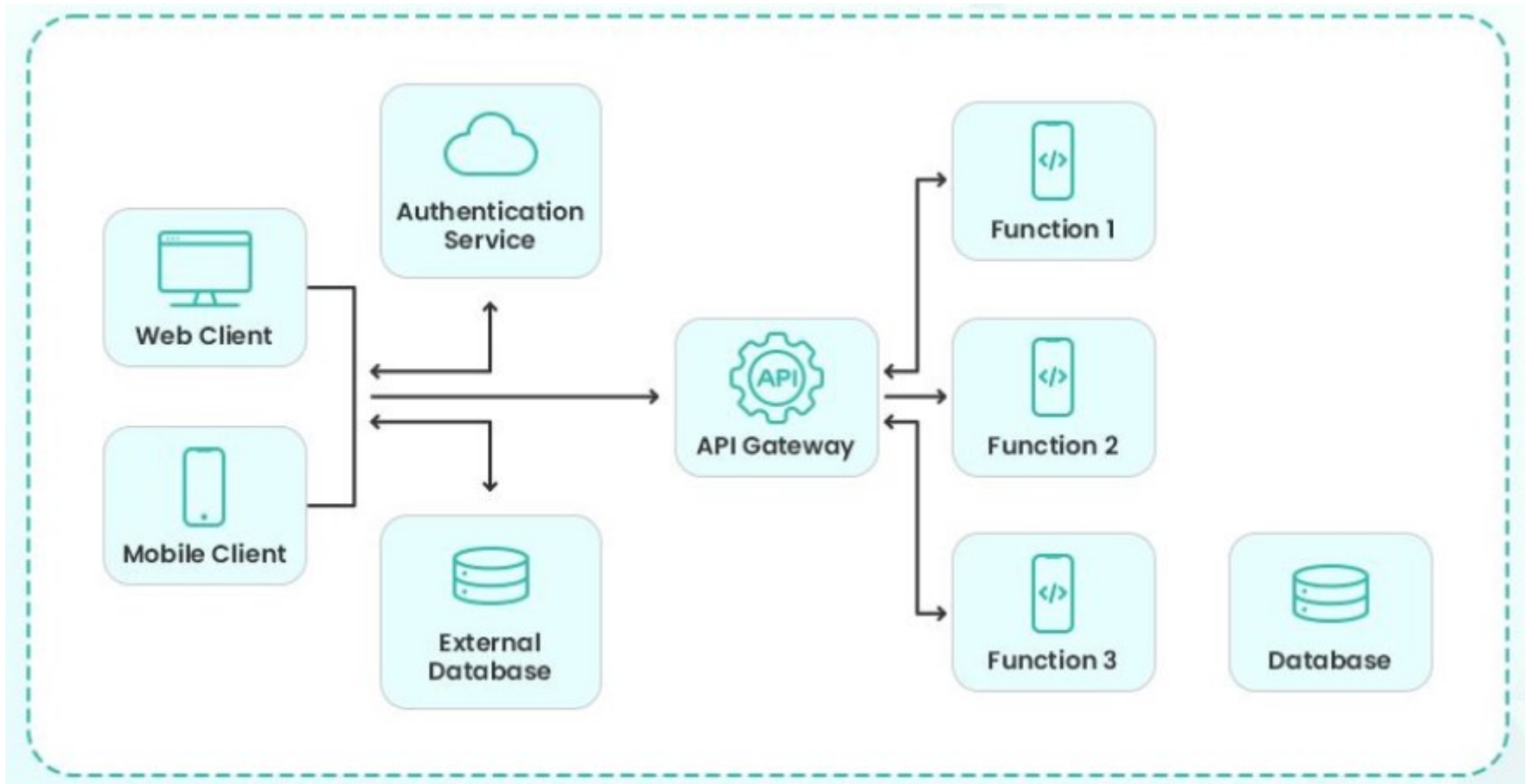
- **Serverless Architecture**

- Is a design pattern where applications are built and run without any manual intervention on the servers that are managed by third-party cloud service providers like Amazon and Microsoft. The entire infrastructure is supported by third-party providers. A third-party provider contributes an outsourcing server and infrastructure management.
- Is categorized into two types:
 - **Backend-as-a-Service** (BaaS): lets developers focus on the frontend development tasks, eliminating the operations performed on the backend. Examples of BaaS providers are Firebase, Azure.
 - **Function-as-a-Service** (FaaS) - an event-driven model: allows developers to break the applications into small functions to focus on the code and event triggers. The rest will be handled by the FaaS service providers such as AWS Lambda, Microsoft Azure Functions, ²⁰ Google Cloud Functions.

Web Application Architecture



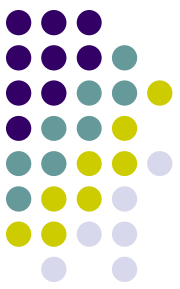
- Serverless Architecture



Aspect	Traditional architecture	Serverless architecture
Infrastructure Management	Developers responsible for provisioning, maintaining, and scaling servers	Cloud providers handle infrastructure management, freeing up developers' time
Resource Allocation	Requires upfront estimation and allocation, can lead to over- or under-provisioning	Dynamically allocates resources based on demand, optimizing resource usage
Scalability	Can be challenging, and often requires manual intervention and configuration changes	Automatically scales based on demand, no manual intervention needed
Cost Model	Fixed cost model, pay for pre-allocated resources regardless of usage	Pay-as-you-go model, pay only for actual compute resources consumed
Deployment Speed	Can be time-consuming, requires managing infrastructure and coordinating deployment	Simplified deployment, developers can deploy individual functions directly to the cloud provider
Flexibility	Can be rigid, may lock into specific technologies, platforms, or configurations	Greater flexibility, supports various programming languages, frameworks, and services

- **Serverless Architecture**

Web Application Architecture

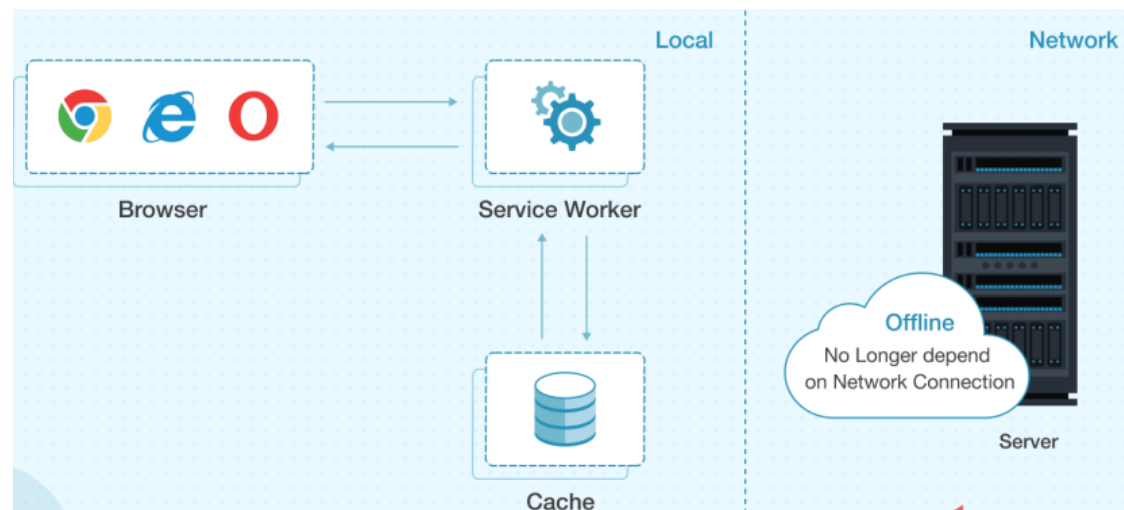


- **Progressive Web Applications (PWAs)**

- Are used to develop apps that offer rich and native functionality with enhanced capabilities, reliability, and easy installation.
- Are compatible apps with any browser and can run on any device.
- Can easily be discovered and shared through URL instead of the app store.
- Work efficiently on poor internet connectivity and in offline mode as well.
- Uber, Aliexpress, Alibaba, Pinterest, and Starbucks use PWAs for developing their products.
- Drawbacks: the need to select the browser and OS that fully support PWA.

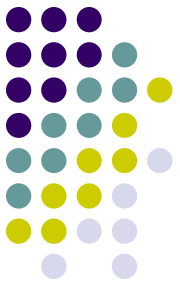
- Service worker acts as a network proxy for the application.

- Cache provides storage for data; catching allows you to refer to the previous result to make a computation faster.

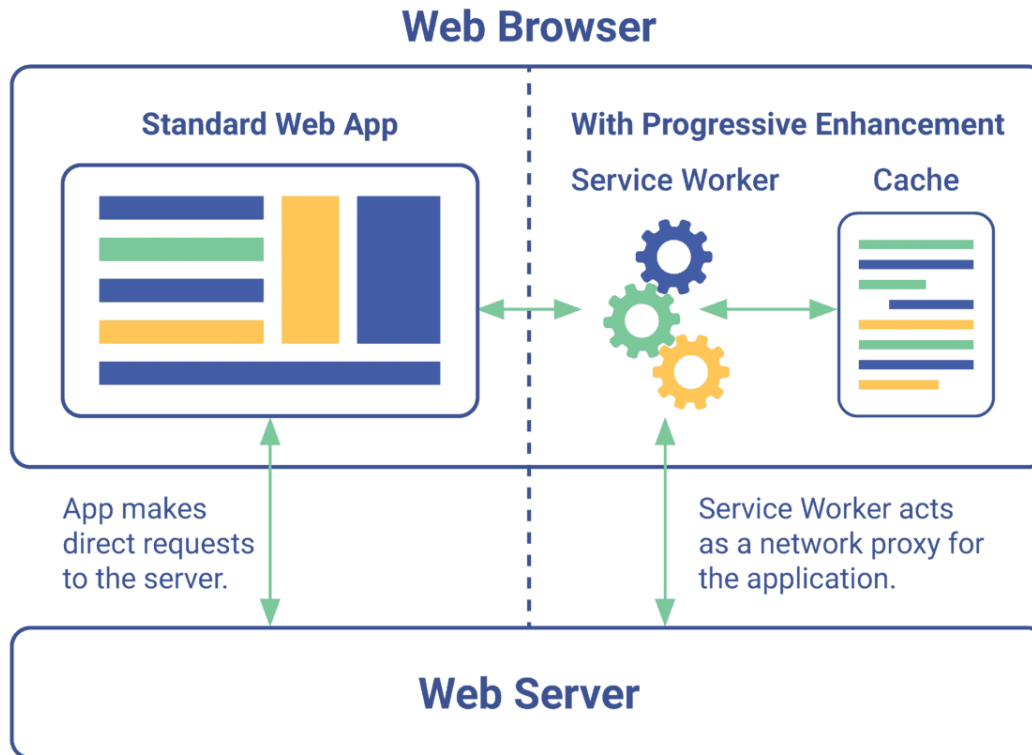


<https://www.simform.com/blog/web-application-architecture/>

Web Application Architecture

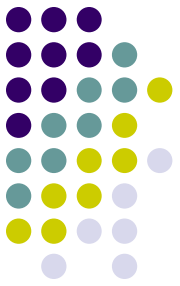


- Progressive Web Applications (PWAs)



<https://softwareplanetgroup.co.uk/dawn-of-the-progressive-web-application/>

Mobile Application Architecture

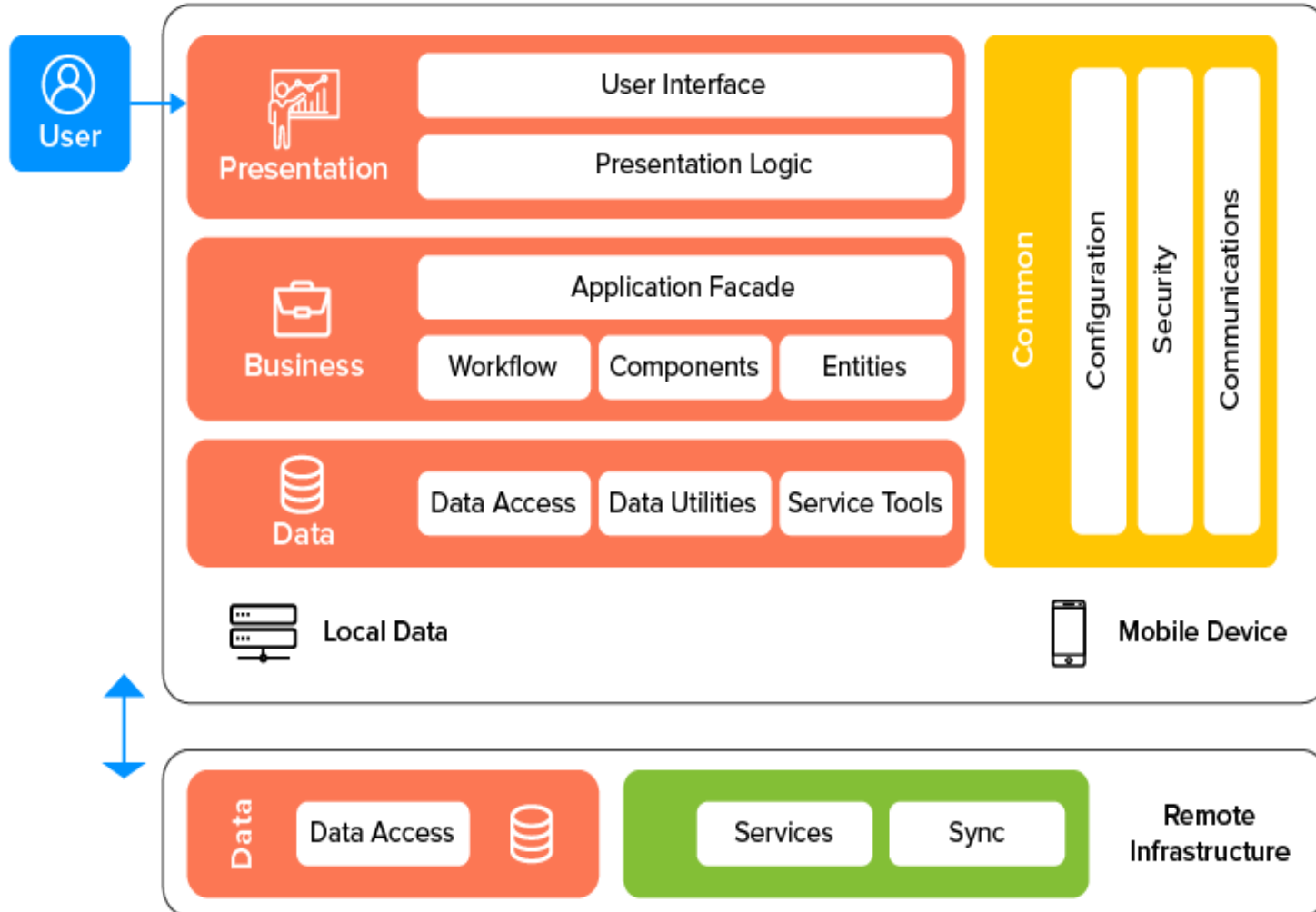


Mobile application architecture

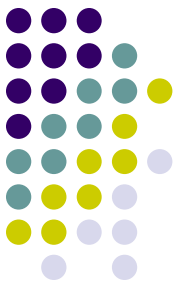
- refers to a **set of rules, techniques, processes, and patterns** to develop a mobile application. These rules help developers create an app that meets both the business requirements as well as industry standards.
- is made up of all the parts of the application – all the questions about **why, what, how** – including what data is collected, how the data moves, what the application looks like, for what platform, using what tech stack.

Mobile Application Architecture

The **three-layer architecture** - The most popular multilayer architecture .



<https://os-system.com/blog/mobile-app-architecture-how-to-design-it/>

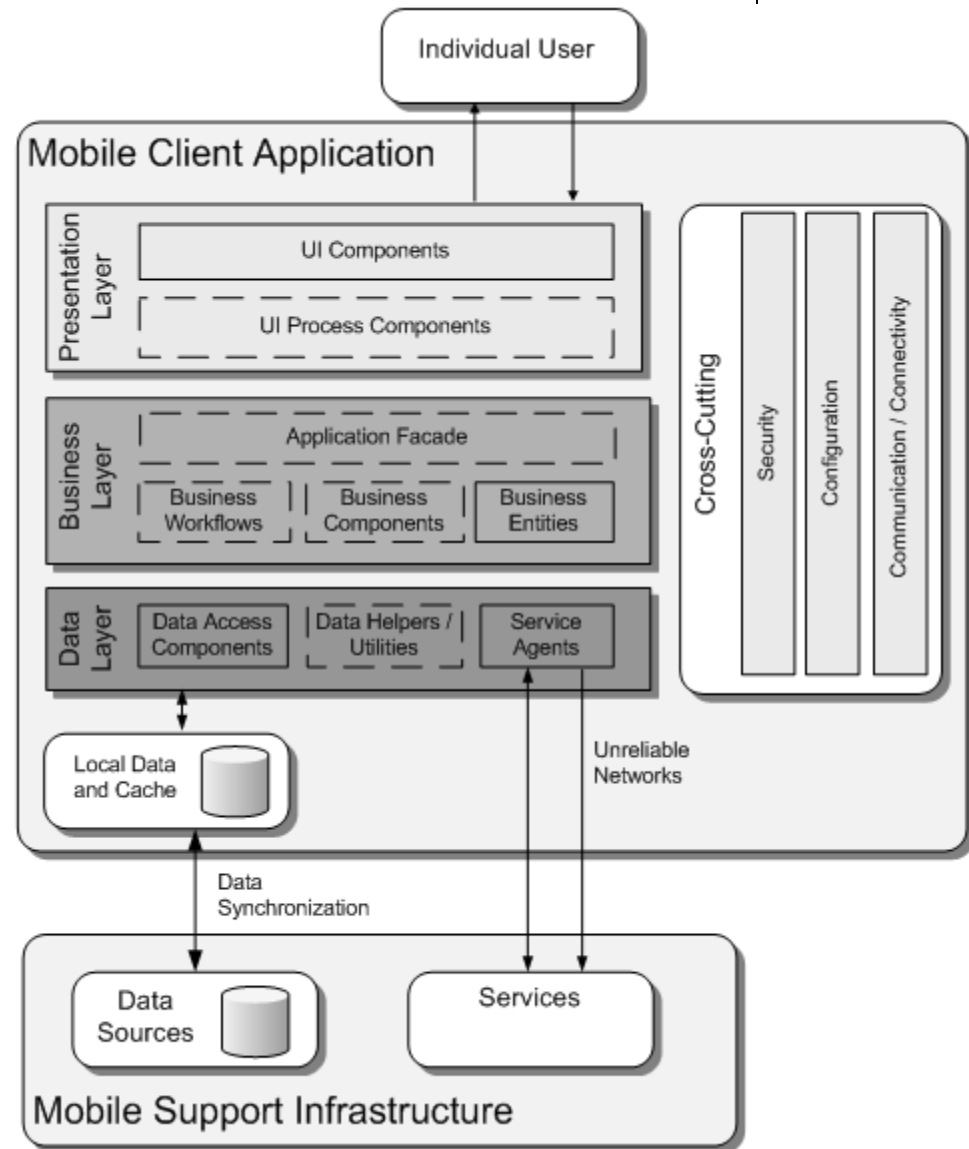


Mobile Application Architecture

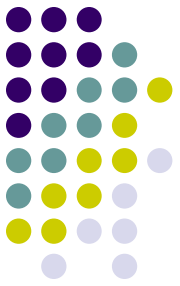


The three-layer architecture

https://guidanceshare.com/wiki/Application_Architecture_Guide_-_Chapter_19_-_Mobile_Applications



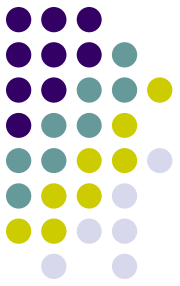
Mobile Application Architecture



Presentation layer

- Consists of all the processes and components to deliver the application to the user.
- Is made up of
 - **User Interface (UI)** is concerned with design questions such as colors, fonts, placement, and overall design.
 - **User Experience (UX)** manages the way a customer interacts with the app through a detailed understanding of what a user wants and feels.

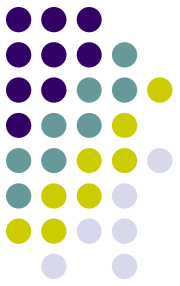
Mobile Application Architecture



Business layer

- Includes **business components**, **workflow**, and **entities**.
- Looks at how the application will present the business to the end-users.
- Is responsible for: security, data caching, logging, data validation, exception management.
- Can exist on the server or on the user device, depending on the operations of the app and the resources each operation takes.

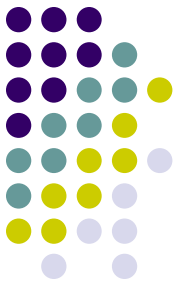
Mobile Application Architecture



Data layer

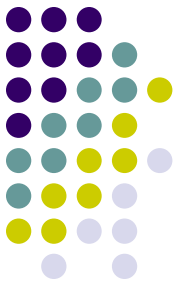
- Includes all the **data utilities**, **service agents**, and **data access** components to support data transactions.
- Can be thought of in two parts:
 - Persistence - data access with data sources via API
 - Network - network communication, routing, error reporting

Mobile Application Architecture



Factors to consider during mobile application development architecture

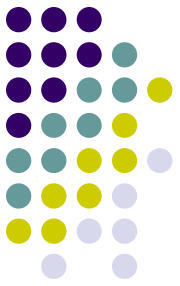
- **Device type**: platform (operating system), different models of smartphone, screen size, screen resolution, CPU (processor), and RAM (memory).
- **Bandwidth Scenarios**: the account power consumption, design access mechanism, speed, the best software protocols and hardware for mobile application.
- **User interface**: themes, fonts, colors, shadings, the way a customer interacts with the application through a detailed understanding of what a user wants and feels.



Mobile Application Architecture

Factors to consider during mobile application development architecture

- **Navigation**: menu; search; bars, rails, drawers, or tabs; familiar icons; intuitive labeling; site organization; gestures; scrolling; thumb zone navigation.
- **Developmental Frameworks**
 - Front-end frameworks: Bootstrap, Foundation, React, Angular, Vue, and Backbone.
 - Back-end (server-side) frameworks: Ruby on Rails, Flask, Django, Laravel, Swift, Xamarin, React Native and Flutter.



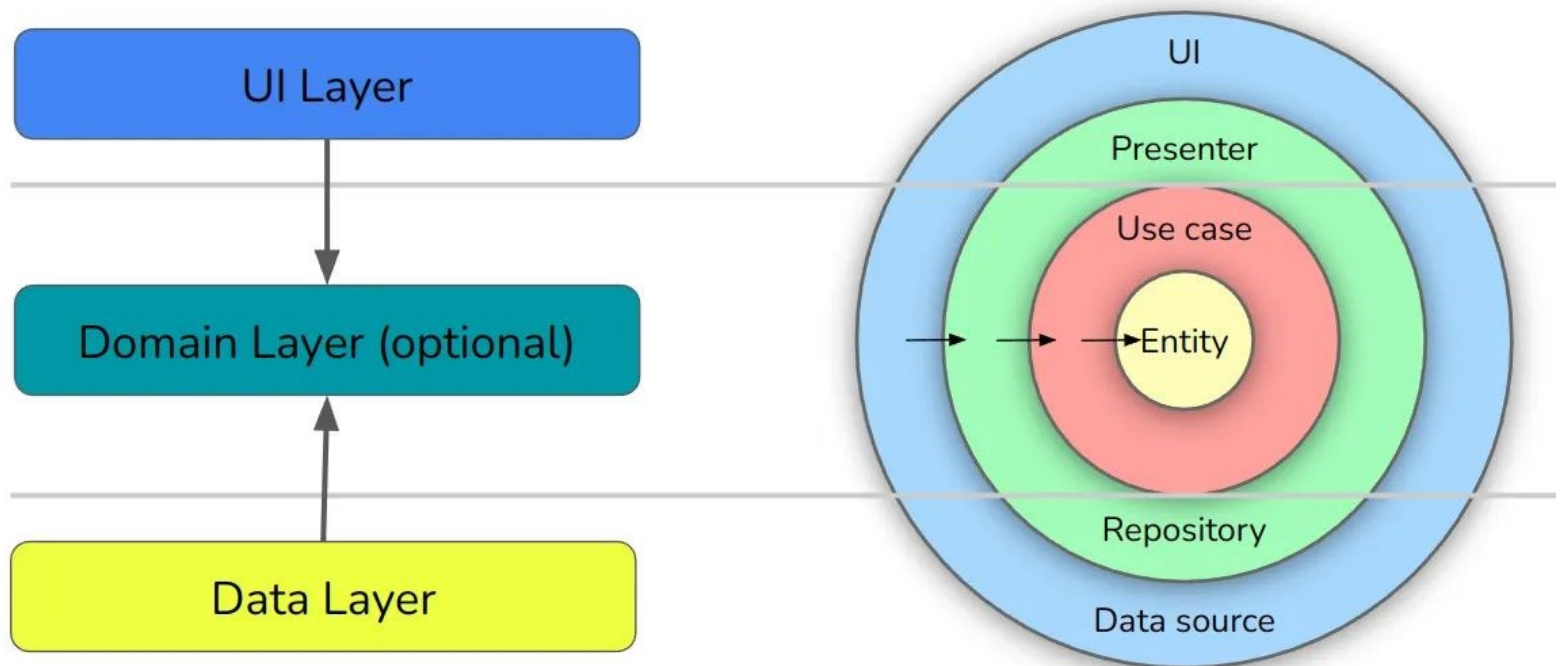
Mobile Application Architecture

- **Android** Mobile Application Architecture.
- **iOS** Mobile Application Architecture
- **Hybrid** Mobile Application Architecture

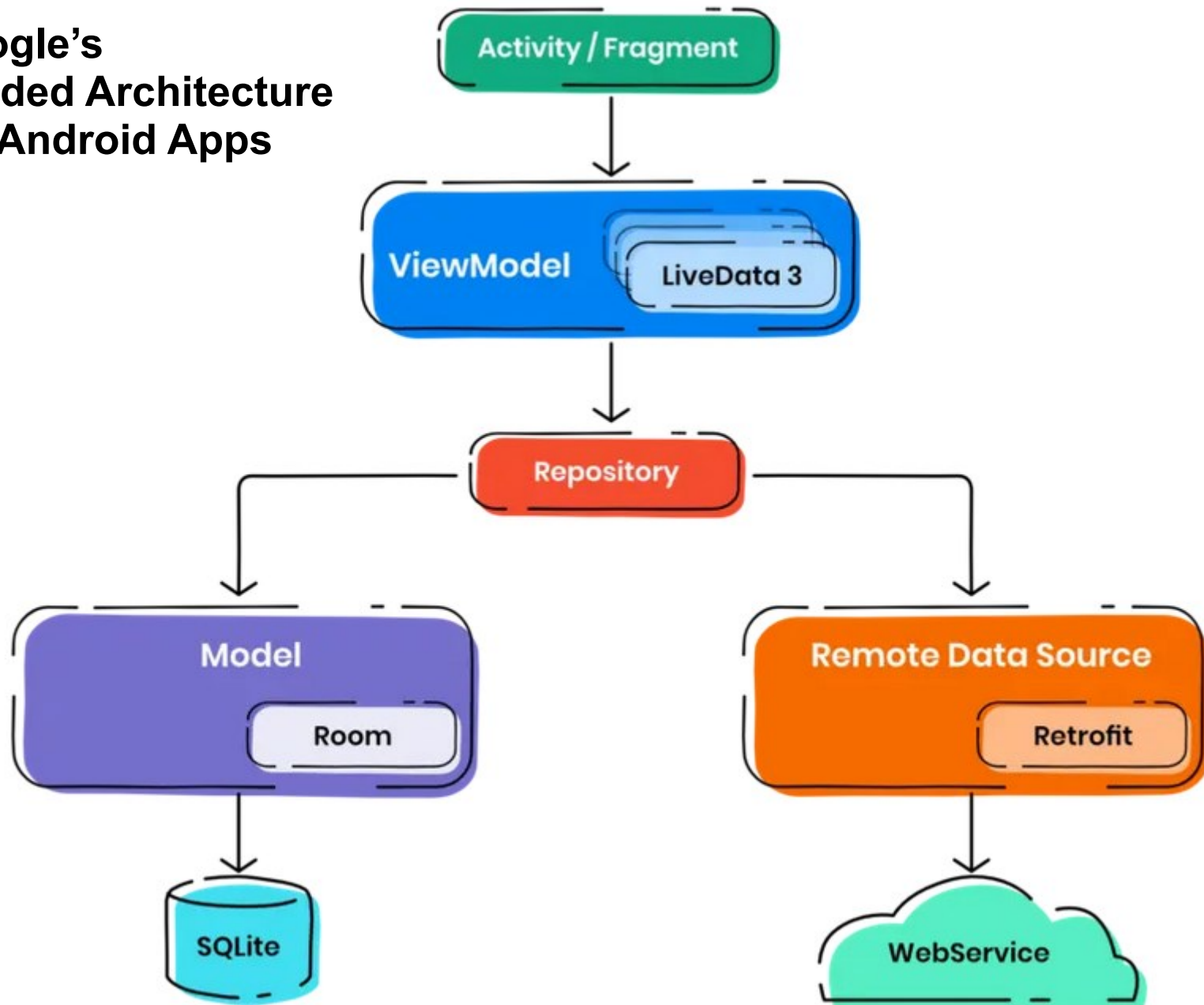
Mobile Application Architecture



- The **Clean Architecture** is the most commonly accepted **Android** architecture for mobile applications.
 - Clean focuses on the same 3 layer structure modeled above, with the business layer sometimes referred to as the domain layer.
 - The domain / business layer must not depend on the other layers, but instead must leverage interfaces.



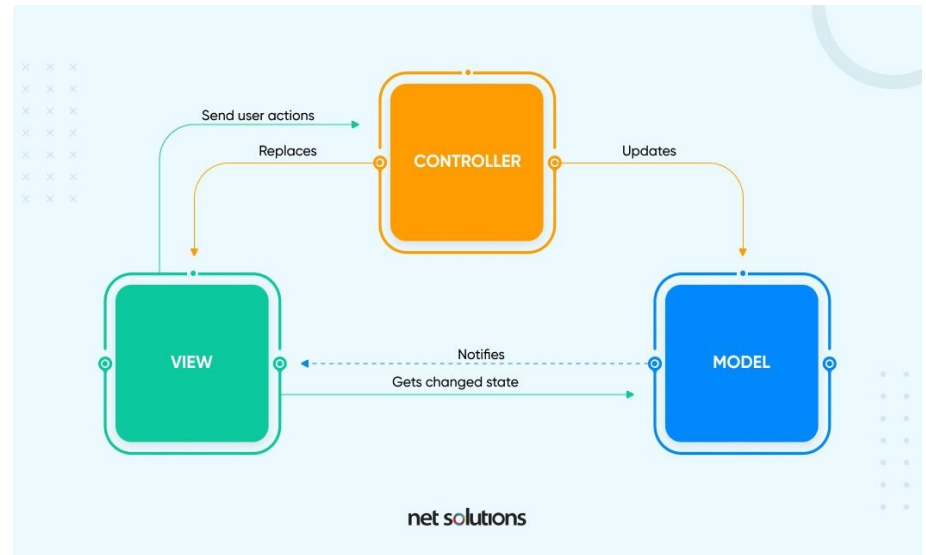
MVVM: Google's Recommended Architecture Pattern for Android Apps



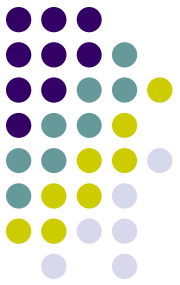
Mobile Application Architecture



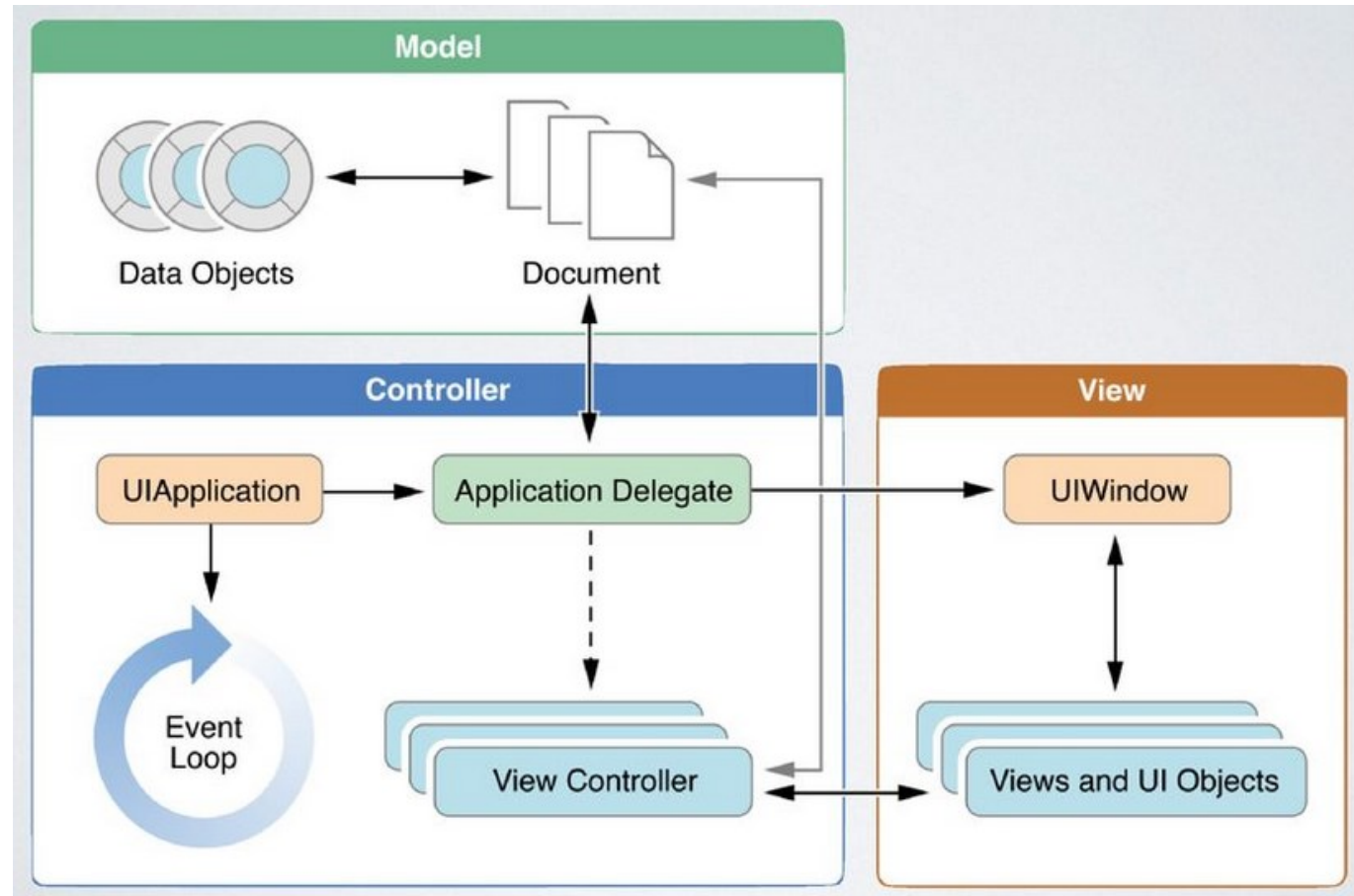
- **iOS** Mobile Application Architecture
 - **Model**: is responsible for managing the data and business logic of app.
 - **View**: is what the user interacts with on the screen. This could be a button, a label, or any other interface element that users can see and touch.
 - **Controller**: acts as a coordinator between the Model and the View. It takes user input from the View, updates the Model accordingly, and manages the communication between the two
- The MVC model supports rapid and parallel development, with the ability to create multiple views – a great option for mobile development.

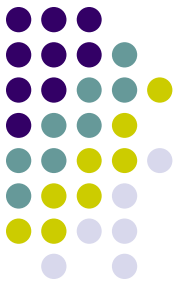


Mobile Application Architecture



- **iOS** Mobile Application Architecture
 - The MVC model supports rapid and parallel development, with the ability to create multiple views – a great option for mobile development.

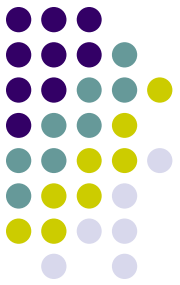




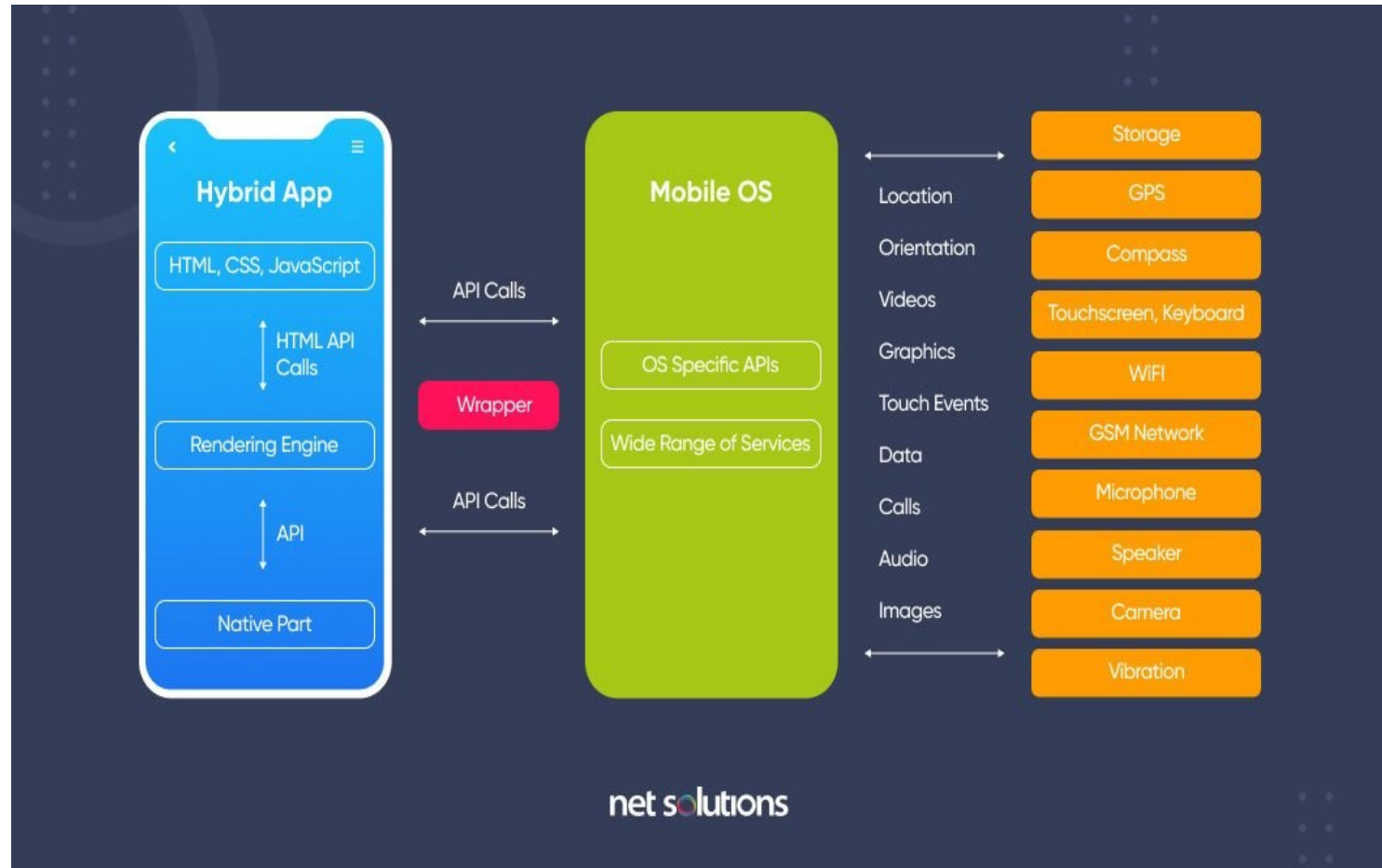
Mobile Application Architecture

- **Hybrid** mobile applications leverage both native and web solutions.
- Hybrid mobile applications use:
 - native applications as “shells” for the back-end,
 - platform-neutral JavaScript, HTML and CSS for the front-end.
 - plugins such as Apache Cordova or Ionic Capacitor to access native platform features.
- Hybrid mobile applications are:
 - among the fastest applications to create across a variety of platforms and easy to update,
 - not appropriate for complex, interactive, or feature-rich applications

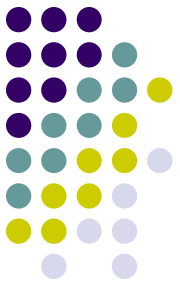
Mobile Application Architecture



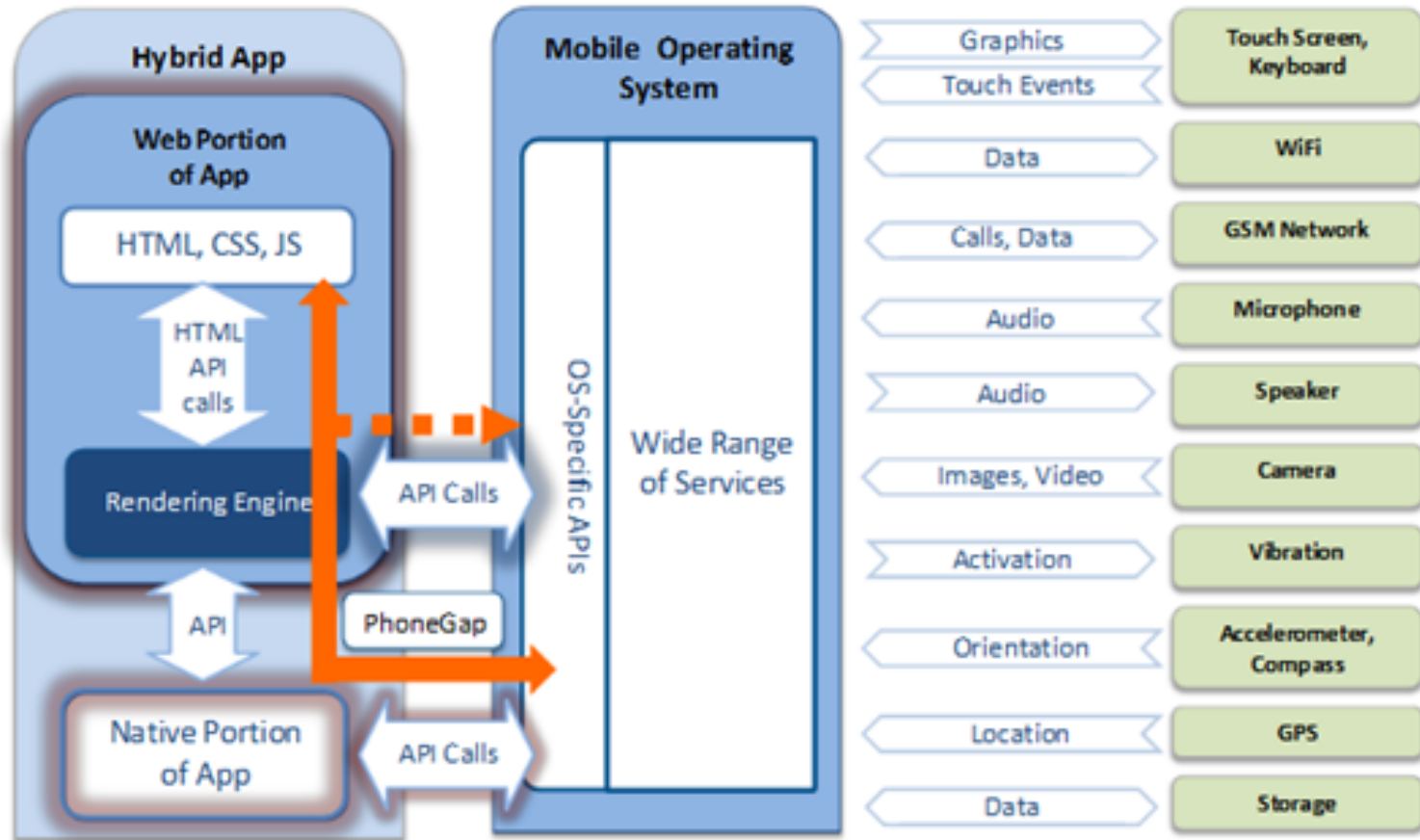
- Hybrid mobile applications



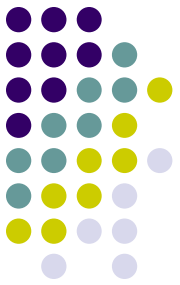
Mobile Application Architecture



- Hybrid mobile applications

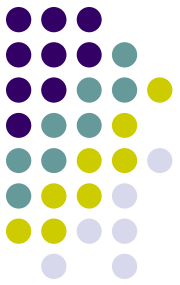


References



- <https://www.simform.com/blog/web-application-architecture/>
- <https://www.intuz.com/guide-on-web-app-architecture>
- <https://litslink.com/blog/web-application-architecture>
- <https://radixweb.com/blog/web-application-architecture-guide>
- <https://radixweb.com/blog/guide-to-web-application-development#web>
- <https://os-system.com/blog/mobile-app-architecture-how-to-design-it/>
- <https://www.netsolutions.com/insights/mobile-app-architecture-guide/>
- https://guidanceshare.com/wiki/Application_Architecture_Guide_-_Chapter_19_-_Mobile_Applications
- <https://docs.wavemaker.com/learn/hybrid-mobile/building-hybrid-mobile-apps/>

Appendix



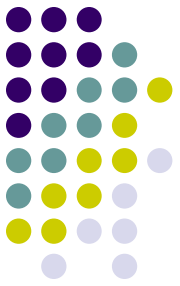
Mobile Applications	Web Applications
Mobile applications are built for a particular operating system, such as iOS, Android, or Windows	Web apps are applications run using internet browsers, most commonly using HTML5, or JavaScript.
Mobile applications are accessible from the Google Play Store and Apple App store.	Browser-based web applications are accessible from web browsers, such as Google Chrome, Mozilla Firefox, or Safari.

Appendix



Website	Web Application
A website is a collection of web pages and related content that is identified by common domain name and published on at least one web server.	The web application is an application program stored on a remote server and delivered over the Internet through a browser interface.
A website is used for communication purposes.	A web application is used for automating tasks to achieve certain goals.
A website allows data visualization (reading and comprehending).	A web application permits data manipulation.
A website is less resource-intensive and needs less processing power.	A web application is more resource-intensive and needs more processing power.

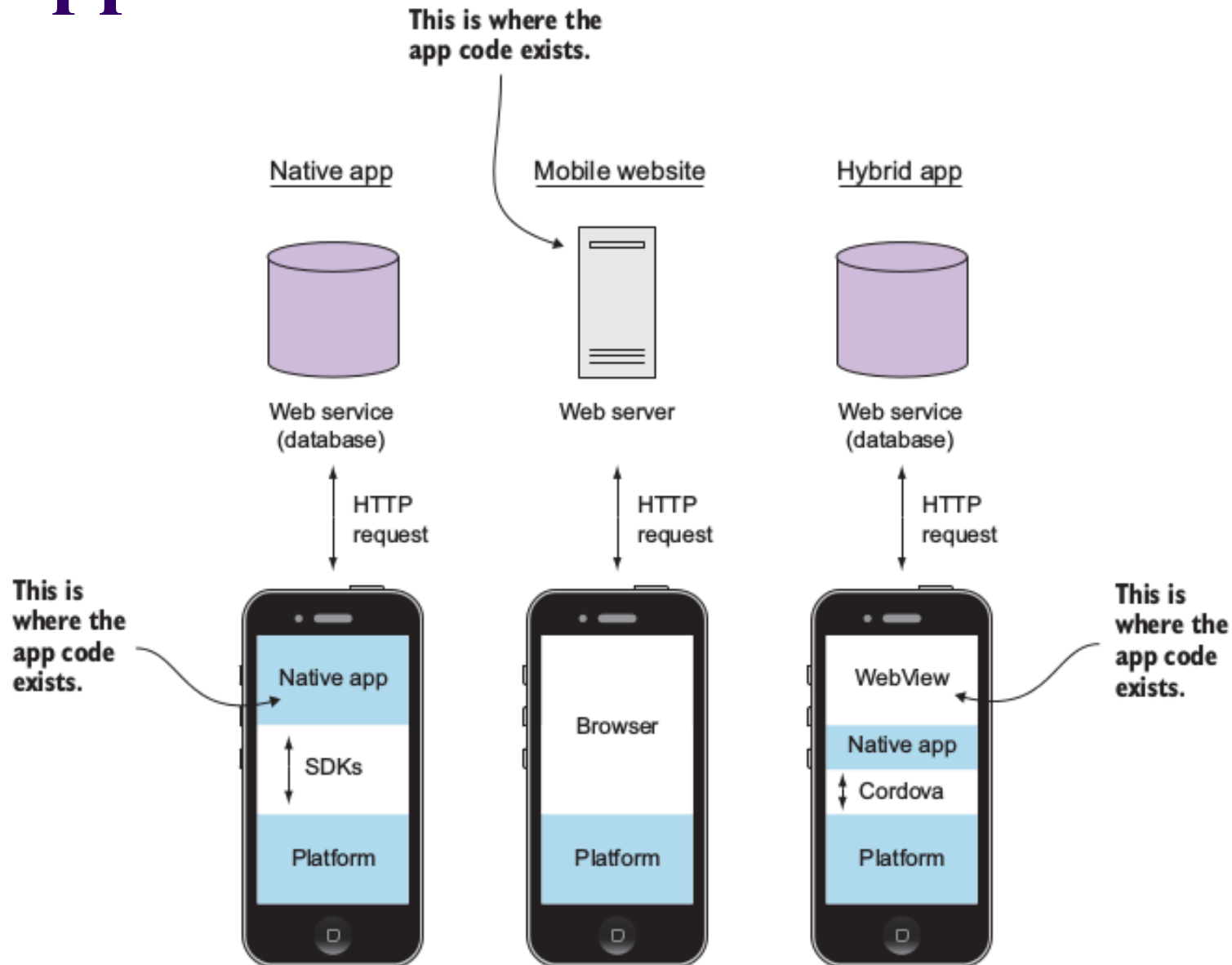
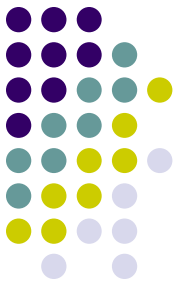
Appendix



Native App	Web App
A software program that is built for a specific platform, such as either iOS or Android, falls under native mobile apps.	A web app that runs in a web browser and is delivered on the World Wide Web to users with an active internet connection.

Appendix

Native App, Web App, Hybrid App



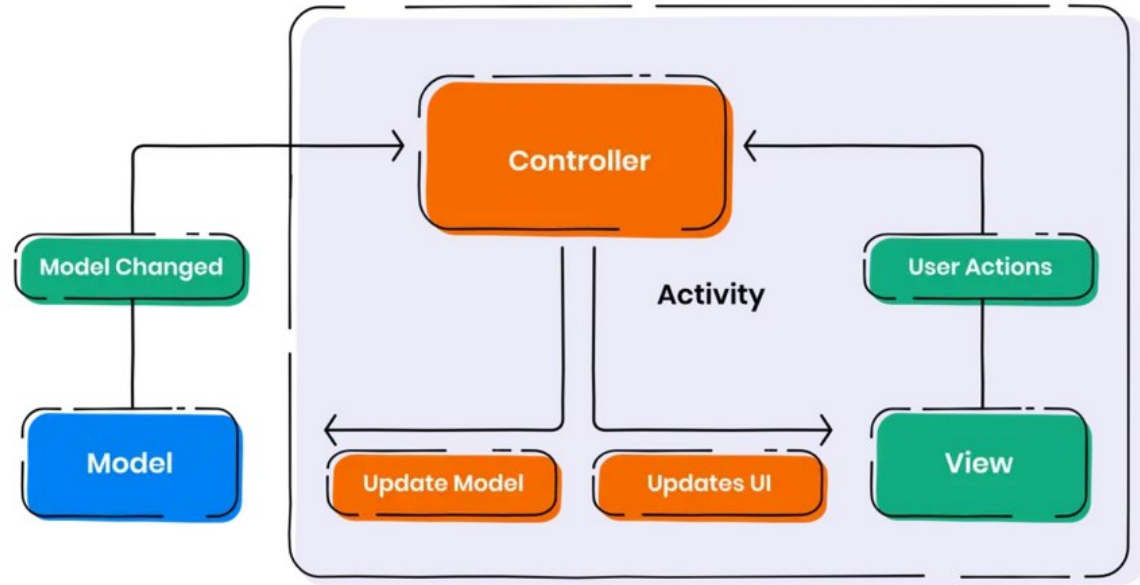
Appendix



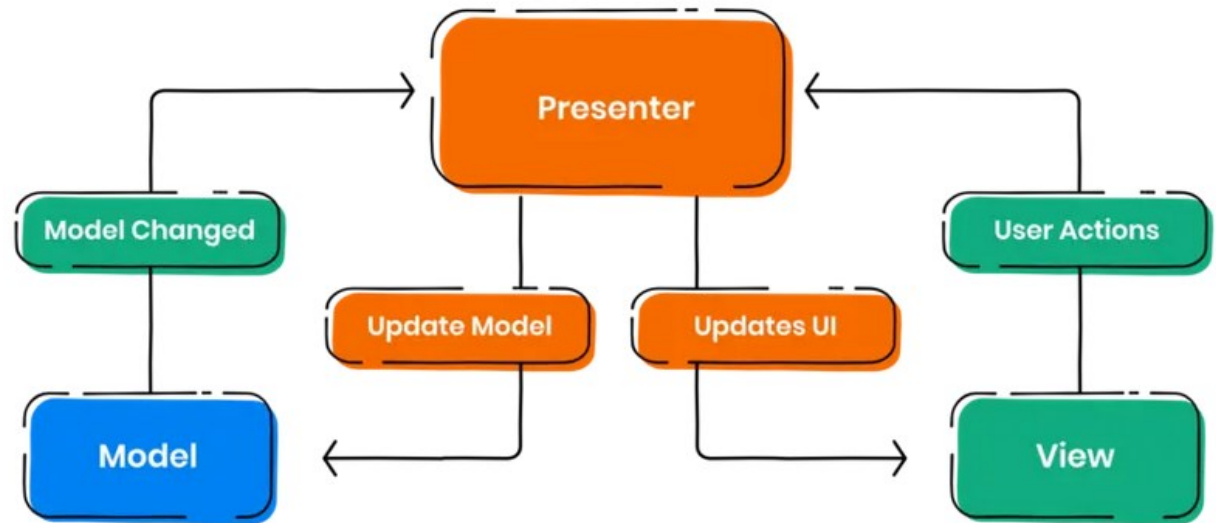
Feature	Native App	Hybrid App	Web App
Device Access	Full	Limited	Full (but with plugins)
Performance	High	Medium to High	Medium to High
Development Language	Swift, Objective-C, Java, Kotlin	HTML, CSS, JavaScript	HTML, CSS, JavaScript
Cross-platform Support	No	Yes	Yes
User Experience	High	Medium to High	Medium to High
Code Reusability	No	Yes	Yes

Popular Android App Architecture Patterns

- View – how to present data,
- Model – how to structure the data,
- Controller – how to handle user interaction.

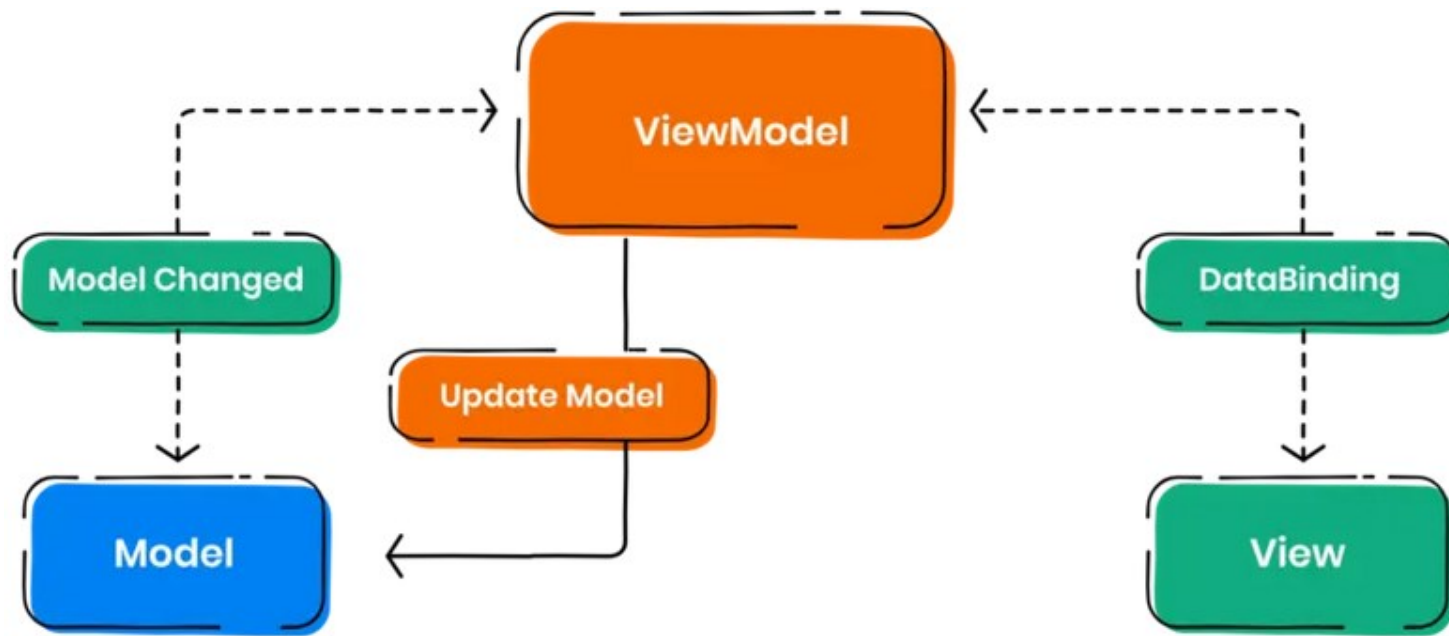


MVC (Model – View – Controller) Architecture



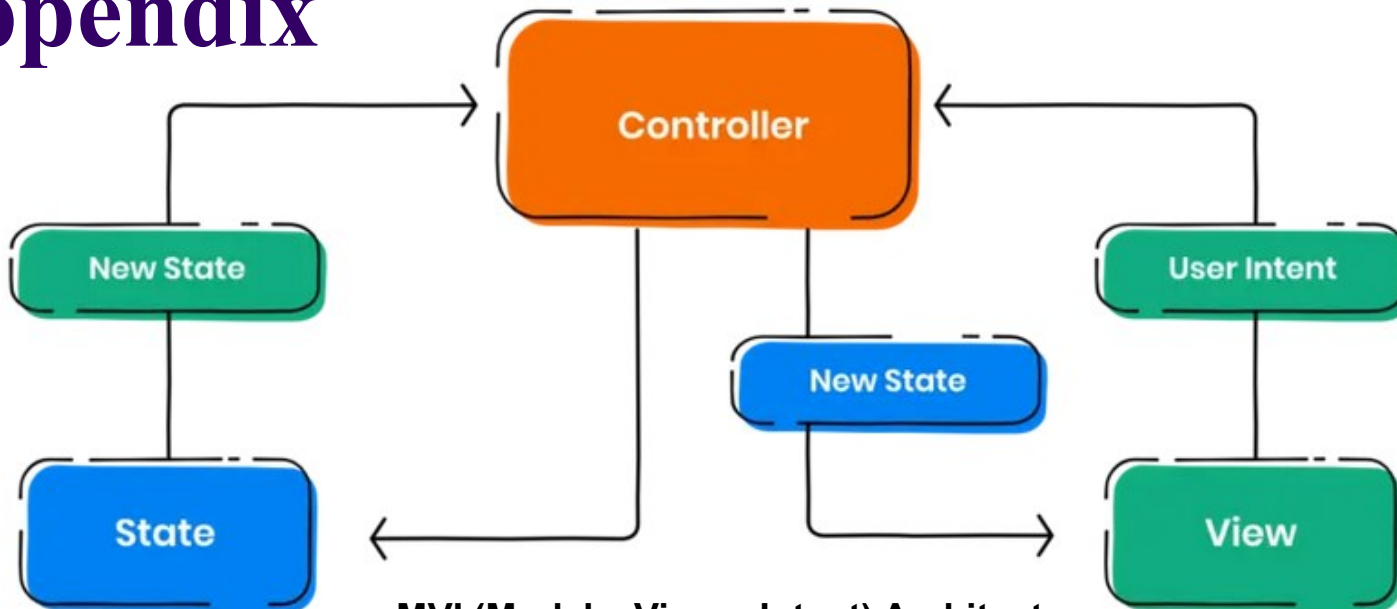
MVP (Model – View – Presenter) Architecture

Appendix

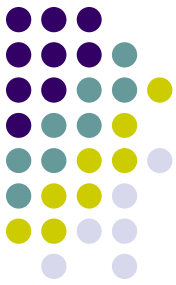


MVVM (Model – View – ViewModel) Architecture

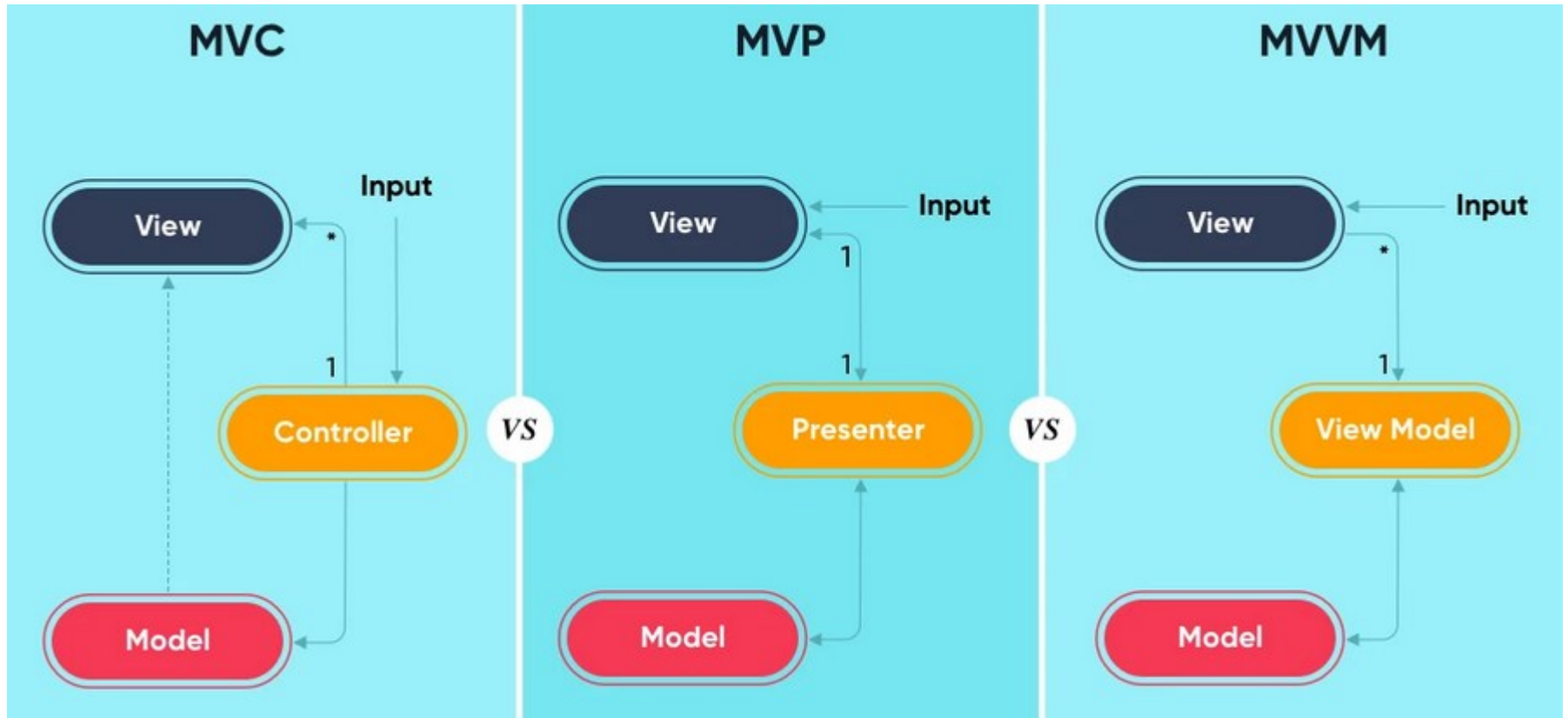
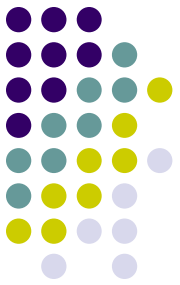
Appendix



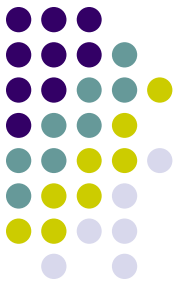
MVI (Model – View – Intent) Architecture



Appendix



<https://www.netsolutions.com/insights/software-design-pattern/>



Appendix

- N-Tier architecture is a physical implementation of N-layer architecture.
- Design pattern
 - A general or reusable solution to a commonly occurring problem in the design of software.
 - A template for how to solve a problem that has been used in many different situations.
 - Not a finished design.
 - MVC is a design pattern.